DATA + AI FOUNDATIONS

What is Retrieval Augmented Generation (RAG)?

Technique enhancing LLM responses by retrieving relevant info from external knowledge bases before generation, grounding outputs in facts

by Databricks Staff

SUMMARY

What Is Retrieval Augmented Generation, or RAG?

Retrieval augmented generation (RAG) is a hybrid AI framework that bolsters large language models (LLMs) by combining them with external, up-to-date data sources. Instead of relying solely on static training data, RAG retrieves relevant documents at query time and feeds them into the model as context. By incorporating new and context-aware data, AI can generate more accurate, current and domain-specific responses.

RAG is quickly becoming the go-to architecture for building enterprise-grade AI applications. According to recent surveys, over 60% of organizations are developing AI-powered retrieval tools to improve reliability, reduce hallucinations and personalize outputs using internal data.

As generative AI expands into business functions like customer service, internal knowledge management and compliance, RAG's ability to bridge the gap between general AI and specific organizational knowledge makes it an essential foundation for trustworthy, real-world deployments.

RAG enhances a language model's output by injecting it with context-aware and real-time information retrieved from an external data source. When a user submits a query, the system first engages the retrieval model, which uses a vector database to identify and "retrieve" semantically similar documents, databases or other sources for relevant information. Once identified, it then combines those results with the original input prompt and sends it to a generative AI model, which synthesizes the new information into its own model.

This allows the LLM to produce more accurate, context-aware answers grounded in enterprise-specific or up-to-date data, rather than simply relying upon the model it was trained on.

RAG pipelines typically involve four steps: document preparation and chunking, vector indexing, retrieval and prompt augmentation. This process flow helps developers update data sources without retraining the model and makes RAG a scalable and cost-effective solution for building LLM applications in domains like customer support, knowledge bases and internal search.

What challenges does the retrieval augmented generation approach solve?

Problem 1: LLM models do not know *your data*

LLMs use deep learning models and train on massive datasets to understand, summarize and generate novel content. Most LLMs are trained on a wide range of public data so one model can respond to many types of tasks or questions. Once trained, many LLMs do not have the ability to access data beyond their training data cutoff point. This makes LLMs static and may cause them to respond incorrectly, give out-of-date answers or hallucinate when asked questions about data they have not been trained on.

Problem 2: AI applications must leverage *custom data* to be effective

For LLMs to give relevant and specific responses, organizations need the model to understand their domain and provide answers from their data vs. giving broad and generalized responses. For example, organizations build customer support bots with LLMs,

How do companies build such solutions without retraining those models?

Solution: Retrieval augmentation is now an industry standard

An easy and popular way to use your own data is to provide it as part of the prompt with which you query the LLM model. This is called retrieval augmented generation (RAG), as you would retrieve the relevant data and use it as augmented context for the LLM. Instead of relying solely on knowledge derived from the training data, a RAG workflow pulls relevant information and connects static LLMs with real-time data retrieval.

With RAG architecture, organizations can deploy any LLM model and augment it to return relevant results for their organization by giving it a small amount of their data without the costs and time of fine-tuning or pretraining the model.

What are the use cases for RAG?

There are many different use cases for RAG. The most common ones are:

1. **Question-and-answer chatbots:** Incorporating LLMs with chatbots allows them to automatically derive more accurate answers from company documents and knowledge bases. Chatbots are used to automate customer support and website lead follow-up to answer questions and resolve issues quickly.

   For instance, Experian, a multinational data broker and consumer credit reporting company, wanted to build a chatbot to serve internal and customer-facing needs. They quickly realized that their current chatbot technologies struggled to scale to meet demand. By building their GenAI chatbot — Latte — on the Databricks Data Intelligence Platform, Experian was able to improve prompt handling and model accuracy, which gave their teams greater flexibility to experiment with different prompts, refine outputs and adapt quickly to evolutions in GenAI technology.

2. **Search augmentation:** Incorporating LLMs with search engines that augment search results with LLM-generated answers can better answer informational queries and make it easier for users to find the information they need to do their jobs.

their questions easily, including HR questions related to benefits and policies and security and compliance questions.

One way this is being deployed is at Cycle & Carriage, a leading automotive group in Southeast Asia. They turned to Databricks Mosaic AI to develop a RAG chatbot that improves productivity and customer engagement by tapping into their proprietary knowledge bases, such as technical manuals, customer support transcripts and business process documents. This made it easier for employees to search for information via natural language queries that deliver contextual, real-time answers.

What are the benefits of RAG?

The RAG approach has a number of key benefits, including:

1. **Providing up-to-date and accurate responses**: RAG ensures that the response of an LLM is not based solely on static, stale training data. Rather, the model uses up-to-date external data sources to provide responses.
2. **Reducing inaccurate responses, or hallucinations**: By grounding the LLM model's output on relevant, external knowledge, RAG attempts to mitigate the risk of responding with incorrect or fabricated information (also known as hallucinations). Outputs can include citations of original sources, allowing human verification.
3. **Providing domain-specific, relevant responses**: Using RAG, the LLM will be able to provide contextually relevant responses tailored to an organization's proprietary or domain-specific data.
4. **Being efficient and cost-effective**: Compared to other approaches to customizing LLMs with domain-specific data, RAG is simple and cost-effective. Organizations can deploy RAG without needing to customize the model. This is especially beneficial when models need to be updated frequently with new data.

When should I use RAG and when should I fine-tune the model?

RAG is the right place to start, being easy and possibly entirely sufficient for some use cases. Fine-tuning is most appropriate in a different situation, when one wants the LLM's behavior to change, or to learn a different "language." These are not mutually exclusive. As a future

response.

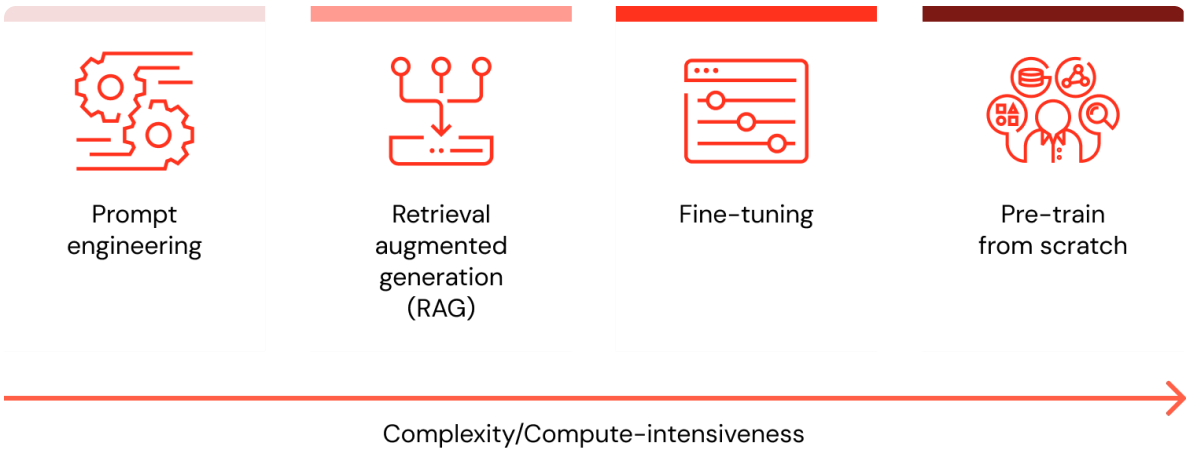
When I want to customize my LLM with data, what are all the options and which method is the best (prompt engineering vs. RAG vs. fine-tune vs. pretrain)?

There are four architectural patterns to consider when customizing an LLM application with your organization's data. These techniques are outlined below and are **not mutually exclusive**. Rather, they can (and should) be combined to take advantage of the strengths of each.

Method	Definition	Primary use case	Data requirements	Advantages	Considerations
 **Prompt engineering**	Crafting specialized prompts to guide LLM behavior	Quick, on-the-fly model guidance	None	Fast, cost-effective, no training required	Less control than fine-tuning
 **Retrieval augmented generation (RAG)**	Combining an LLM with external knowledge retrieval	Dynamic datasets and external knowledge	External knowledge base or database (e.g., vector database)	Dynamically updated context, enhanced accuracy	Increases prompt length and inference computation

Fine-tuning	Adapting a pretrained LLM to specific datasets or domains	Domain or task specialization	Thousands of domain-specific or instruction examples	Granular control, high specialization	Requires labeled data, computational cost
Pretraining	Training an LLM from scratch	Unique tasks or domain-specific corporation	Large datasets (billions to trillions of tokens)	Maximum control, tailored for specific needs	Extremely resource-intensive

Regardless of the technique selected, building a solution in a well-structured, modularized manner ensures organizations will be prepared to iterate and adapt. Learn more about this approach and more in The Big Book of MLOps.



Common challenges in RAG implementation

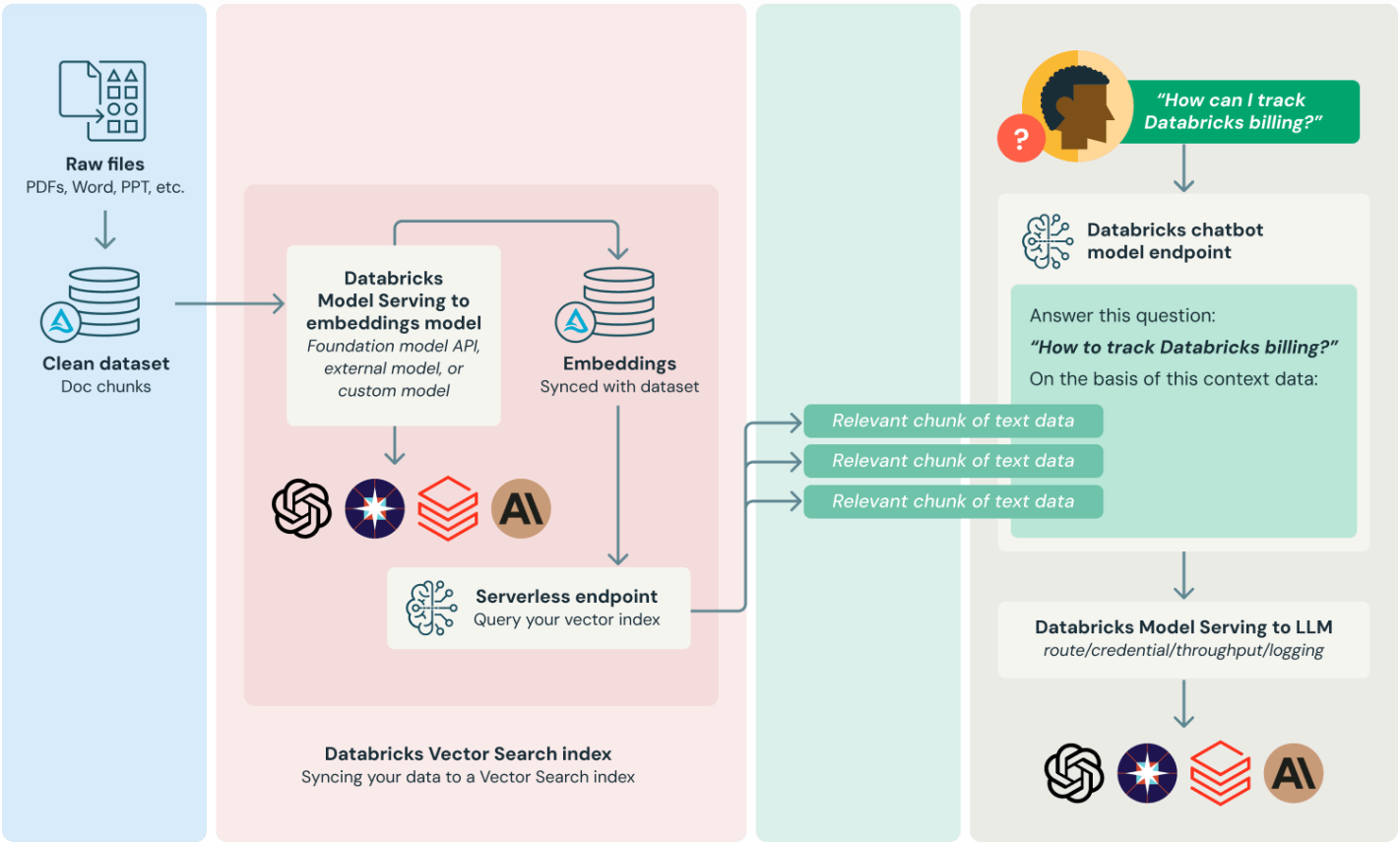
Implementing RAG at scale introduces several technical and operational challenges.

effective retrieval pipeline that includes careful selection of embedding models, similarity metrics and ranking strategies.

2. **Context window limitations.** With the entirety of the world's documentation at its fingertips, the risks can be injecting too much content into the model, leading to truncated sources or diluted responses. Chunking strategies should weight semantic coherence with token efficiency.
3. **Data freshness.** The benefit of RAG is in its ability to cull up-to-date information. However, document indexes can quickly go stale without scheduled ingestion jobs or automated updates. By ensuring your data is fresh, you can avoid hallucinations or outdated answers.
4. **Latency.** When dealing with large datasets or external APIs, latency can interfere with retrieval, ranking and generation.
5. **RAG evaluation.** Because of RAG's hybrid nature, traditional AI evaluation models fall short. Evaluating the accuracy of outputs requires a combination of human judgment, relevance scoring and groundedness checks to assess response quality.

What is a reference architecture for RAG applications?

There are many ways to implement a retrieval augmented generation system, depending on specific needs and data nuances. Below is one commonly adopted workflow to provide a foundational understanding of the process.



1. **Prepare data**: Document data is gathered alongside metadata and subjected to initial preprocessing — for example, PII handling (detection, filtering, redaction, substitution). To be used in RAG applications, documents need to be chunked into appropriate lengths based on the choice of embedding model and the downstream LLM application that uses these documents as context.
2. **Index relevant data**: Produce document embeddings and hydrate a Vector Search index with this data.
3. **Retrieve relevant data**: Retrieving parts of your data that are relevant to a user's query. That text data is then provided as part of the prompt that is used for the LLM.
4. **Build LLM applications**: Wrap the components of prompt augmentation and query the LLM into an endpoint. This endpoint can then be exposed to applications such as Q&A chatbots via a simple REST API.

Databricks also recommends some key architectural elements of a RAG architecture:

To ensure that the deployed language model has access to up-to-date information, regular vector database updates can be scheduled as a job. Note that the logic to retrieve from the vector database and inject information into the LLM context can be packaged in the model artifact logged to MLflow using MLflow LangChain or PyFunc model flavors.

- **MLflow LLM Deployments or Model Serving**: In LLM-based applications where a third-party LLM API is used, the MLflow LLM Deployments or Model Serving support for external models can be used as a standardized interface to route request from vendors such as OpenAI and Anthropic. In addition to providing an enterprise-grade API gateway, the MLflow LLM Deployments or Model Serving centralizes API key management and provides the ability to enforce cost controls.
- **Model Serving**: In the case of RAG using a third-party API, one key architectural change is that the LLM pipeline will make external API calls, from the Model Serving endpoint to internal or third-party LLM APIs. It should be noted that this adds complexity, potential latency and another layer of credential management. By contrast, in the fine-tuned model example, the model and its model environment will be deployed.

Resources

- Databricks blog posts
  - Using MLflow AI Gateway and Llama 2 to Build Generative AI Apps
  - Best Practices for LLM Evaluation of RAG Applications
- Databricks Demo
- Databricks eBook — The Big Book of MLOps

Databricks customers using RAG

JetBlue

JetBlue has deployed "BlueBot," a chatbot that uses open source generative AI models complemented by corporate data, powered by Databricks. This chatbot can be used by all teams at JetBlue to get access to data that is governed by role. For example, the finance team can see data from SAP and regulatory filings, but the operations team will only see maintenance information.